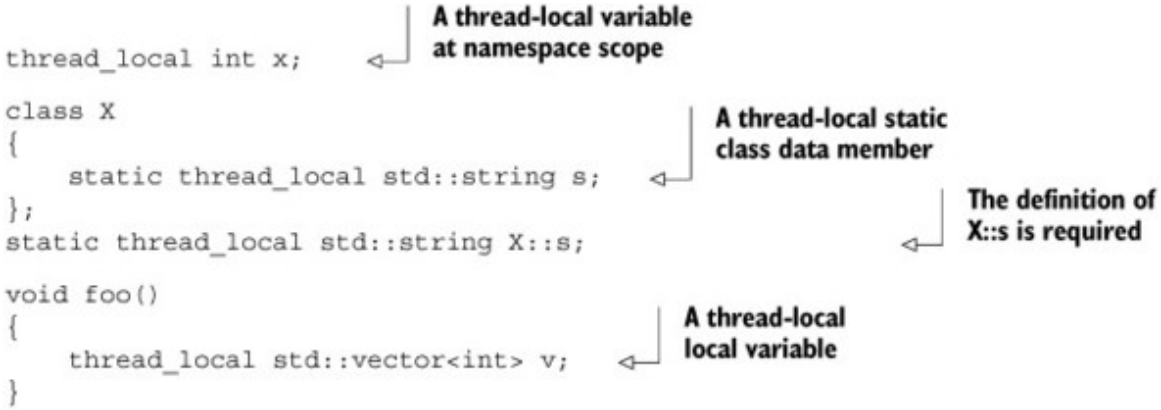
A.8. Thread-local variables

Thread-local variables allow you to have a separate instance of a variable for each thread in your program. You mark a variable as being thread-local by declaring it with the `thread_local` keyword. Variables at namespace scope, static data members of classes, and local variables can be declared thread-local, and are said to have *thread storage duration*:



Thread-local variables at namespace scope and thread-local static class data members are constructed before the first use of a thread-local variable from the same translation unit, but it isn't specified *how much* before. Some implementations may construct thread-local variables when the thread is started; others may construct them immediately before their first use on each thread, and others may construct them at other times, or in some combination depending on their usage context. Indeed, if none of the thread-local variables from a given translation unit is used, there's no guarantee that they will be constructed at all. This allows for the dynamic loading of modules containing thread-local variables—these variables can be constructed on a given thread the first time that thread references a thread-local variable from the dynamically-loaded module.

Thread-local variables declared inside a function are initialized the first time the flow of control passes through their declaration on a given thread. If the function is not called by a given thread, any thread-local variables declared in that function are not constructed. This is just the same as the behaviour for local static variables, except it applies separately to each thread.